FEU INSTITUTE OF TECHNOLOGY

COLLEGE OF COMPUTER STUDIES AND MULTIMEDIA ARTS

CS0065
(INTELLIGENT SYSTEMS)

TECHNICAL ASSESSMENT

4

IMPLEMENTING KNOWLEDGE REPRESENTATION, RULE-BASED REASONING (RBR), AND CASE-BASED REASONING (CBR)

Student Name	Patricia Nicole C. Formoso
Section:	AN41
Professor:	Ms. Crisola Tan
Date:	October 3, 2026

I. BACKGROUND INFORMATION

Knowledge Representation (KR) is the method of organizing facts, rules, and relationships so an AI system can understand and reason about the world.

Reasoning (R) allows the system to infer, make decisions, and solve problems based on that knowledge. Two major reasoning approaches from the module:

1. Rule-Based Reasoning (RBR)

Uses IF–THEN rules, a Knowledge Base, Working Memory, and an Inference Engine.

2. Case-Based Reasoning (CBR)

Solves new problems using past cases using the 4R Cycle:

Retrieve → Reuse → Revise → Retain

This laboratory lets you build small working implementations of both reasoning methods.

II. TECHNICAL ASSESSMENT

IMPLEMENTING KNOWLEDGE REPRESENTATION, RULE-BASED REASONING (RBR), AND CASE-BASED REASONING (CBR)

Objective

At the end of this laboratory activity, students are expected to:

1. Construct a simple knowledge base using Python data structures.
2. Implement a basic Rule-Based Reasoning (RBR) system using IF–THEN logic.
3. Build a simple Case-Based Reasoning (CBR) system using similarity matching.
4. Compare how RBR and CBR behave when solving the same type of problem.
5. Document observations and insights on KR, RBR, and CBR.

Materials Needed

1. Hardware
   - A laptop or desktop computer
   - Stable internet connection (for research, documentation, submission)
2. Software

A programming environment:

- Python (recommended: IDLE, VS Code, Jupyter Notebook, PyCharm)
- Any text editor (Notepad++, Sublime, VS Code) if needed

3. File Formats & Tools

Word processor for documentation:

- MS Word, Google Docs, or LibreOffice
- Screenshot tool (e.g., Snipping Tool, Lightshot, Print Screen)

General Instructions

1. This **Technical Assessment (TA3)** must be completed by Pair.
2. Include clear comments in your code to describe its logic.
3. Each program should display the correct input, process, and output.
4. Ensure that your script displays clear and readable outputs.
5. Screenshot of program output for RBR
6. Screenshot of program output for CBR
7. Reflection answers
8. Short description of your knowledge base and use case
9. Compile all Python codes and corresponding outputs into one PDF file.
10. Save it as **TA4_lastname1.pdf**
    (Example: TA2_Patilano.pdf)
11. Submit the **python file** and **PDF** file.

LABORATORY TASKS

PART 1 — Knowledge Representation (KR) Setup

Task 1A — Create a Knowledge Base

Create a Python dictionary storing at least:

- 5 facts about an object (e.g., weather, appliances, students, medical symptoms)
- 3 categories (e.g., temperature, status, presence, actions)

Example structure to follow:

```
facts = {
    "temperature": 32,
    "is_raining": False,
    "humidity": 85,
}
```

PART 2 — Implement Rule-Based Reasoning (RBR)

Task 2A — Create at least 3 IF–THEN rules

Write Python functions representing rules.

Example:

```
def rule_hot(facts):
    if facts["temperature"] > 30:
        return "Turn on the aircon"
```

Task 2B — Implement an Inference Engine

Create a loop that:

1. Reads the knowledge base
2. Checks each rule
3. Returns all actions inferred

Task 2C — Display Output

Example [Program should print:]

```
Rule-Based Reasoning Actions:
- Turn on the aircon
- Bring umbrella
```

PART 3 — Implement Case-Based Reasoning (CBR)

Task 3A — Build a Case Base

Create at least **3 past cases**, each containing:

- A set of problem features
- A matching solution

Example:

```
case_base = [
    {"problem": {"won't turn on": True, "noise": False},
     "solution": "Check power cable."}
]
```

Task 3C — Reuse Step

Return the solution of the most similar case.

Task 3D — Revise Step

Allow student to adjust solution manually or return unchanged.

Task 3E — Retain Step

Add the new case to the case base.

PART 4 — Comparative Reflection

Write a short paragraph answering:

1. Which system (RBR or CBR) was easier to implement? Why?
   - I found the Rule-Based Reasoning (RBR) system easier to implement because it simply uses standard "IF THEN rules" that map perfectly to basic Python conditionals. Case-Based Reasoning (CBR) required a bit more logic to loop through and calculate similarity scores to find a "matching solution"
2. Which is more flexible in solving unstructured problems?
   - CBR is more flexible for unstructured issues. Because RBR relies strictly on an "Inference Engine" and predefined rules, it breaks if it encounters a scenario it hasn't been explicitly programmed for. CBR just "solves new problems using past cases," so it can guess the best approach even if the new problem isn't a 100% exact match.
3. Which is better for situations with continuously changing data?
   - CBR is better for changing data because it actively learns over time. The "4R Cycle" includes a "Retain Step" that allows you to "Add the new case to the case base". This means the system organically grows smarter with every interaction, whereas RBR would require a developer to constantly hardcode new rules.

4. Examples of real-world applications for both.
   - RBR is great for automated customer service chatbots or simple tax calculation software where the logic is strict and unbending. CBR is ideal for medical diagnosis or technical helpdesks, where professionals look at "past cases" (like patient histories or previous IT tickets) to find a "matching solution" for a new, slightly different issue.

Your Code - Laboratory Tasks ⇩

CODE:

```
# PART 1 - Knowledge Representation

knowledge_base = {
    "facts": {
        "wont_turn_on": True,
        "beep_codes": 3,
        "screen_is_black": True,
        "is_plugged_in": True,
        "spilled_water": False
    },

    "categories": {
        "status": ["wont_turn_on"],
        "error": ["beep_codes"],
        "display": ["screen_is_black"],
        "power": ["is_plugged_in"],
        "damage": ["spilled_water"]
    }
}

facts = knowledge_base["facts"]

print("INPUT - Knowledge Base")
print("Facts:", facts)
print("Categories:", list(knowledge_base["categories"].keys()))


# PART 2 - Rule-Based Reasoning

def rule_power(facts):
    if facts["wont_turn_on"] and not facts["is_plugged_in"]:
        return "Plug in the power cable."
    return None
```

```
def rule_ram(facts):
    if facts["beep_codes"] == 3:
        return "Reseat the RAM sticks."
    return None


def rule_screen(facts):
    if facts["screen_is_black"] and facts["is_plugged_in"]:
        return "Check the monitor display cable."
    return None


rules = [rule_power, rule_ram, rule_screen]


def inference_engine(current_facts):
    actions = []

    print("\nPROCESS - Checking the rules")

    for rule in rules:
        result = rule(current_facts)
        print("Checking", rule.__name__)

        if result:
            print("Action found:", result)
            actions.append(result)
        else:
            print("No action from this rule.")

    return actions


rbr_actions = inference_engine(facts)

print("\nOUTPUT - Rule-Based Reasoning Actions")
for action in rbr_actions:
    print("-", action)


# PART 3 - Case-Based Reasoning
```

```
case_base = [
    {
        "problem": {
            "wont_turn_on": True,
            "beep_codes": 0
        },
        "solution": "Check the power cable."
    },
    {
        "problem": {
            "wont_turn_on": True,
            "beep_codes": 3
        },
        "solution": "Reseat the RAM sticks."
    },
    {
        "problem": {
            "screen_is_black": True,
            "beep_codes": 1
        },
        "solution": "Replace the monitor."
    }
]

new_problem = {
    "wont_turn_on": True,
    "beep_codes": 3,
    "screen_is_black": True
}

print("\nINPUT - New Problem")
print(new_problem)


# Retrieve step
def get_similarity(past_problem, current_problem):
    score = 0

    for key in current_problem:
        if key in past_problem:
            if past_problem[key] == current_problem[key]:
                score += 1
```

```
    return score


print("\nPROCESS - Comparing the new problem with past cases")

best_match = None
highest_score = -1

for case in case_base:
    score = get_similarity(case["problem"], new_problem)
    print("Similarity score:", score)

    if score > highest_score:
        highest_score = score
        best_match = case


# Reuse step
final_solution = best_match["solution"]

print("\nREUSE - Retrieved Solution")
print(final_solution)

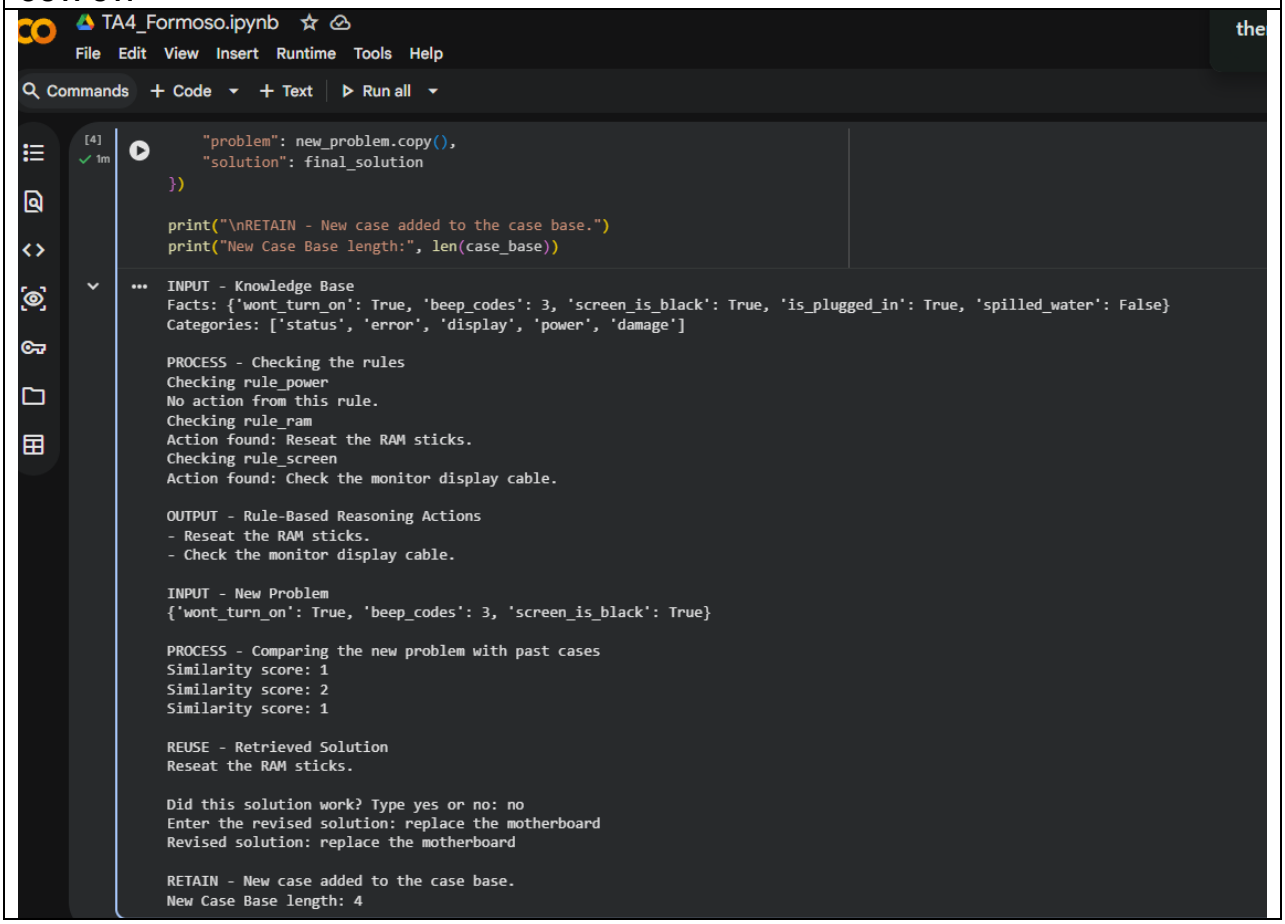

# Revise step
user_feedback = input("\nDid this solution work? Type yes or no:
").strip().lower()

if user_feedback == "no":
    final_solution = input("Enter the revised solution: ").strip()
    print("Revised solution:", final_solution)
else:
    print("The retrieved solution was kept.")


# Retain step
case_base.append({
    "problem": new_problem.copy(),
    "solution": final_solution
})

print("\nRETAIN - New case added to the case base.")
print("New Case Base length:", len(case_base))
```

OUTPUT:


TA4_Formoso.ipynb
File Edit View Insert Runtime Tools Help
Commands + Code + Text Run all
[4]
1m
"problem": new_problem.copy(),
"solution": final_solution
})
print("\nRETAIN - New case added to the case base.")
print("New Case Base length:", len(case_base))
INPUT - Knowledge Base
Facts: {'wont_turn_on': True, 'beep_codes': 3, 'screen_is_black': True, 'is_plugged_in': True, 'spilled_water': False}
Categories: ['status', 'error', 'display', 'power', 'damage']
PROCESS - Checking the rules
Checking rule_power
No action from this rule.
Checking rule_ram
Action found: Reseat the RAM sticks.
Checking rule_screen
Action found: Check the monitor display cable.
OUTPUT - Rule-Based Reasoning Actions
- Reseat the RAM sticks.
- Check the monitor display cable.
INPUT - New Problem
{'wont_turn_on': True, 'beep_codes': 3, 'screen_is_black': True}
PROCESS - Comparing the new problem with past cases
Similarity score: 1
Similarity score: 2
Similarity score: 1
REUSE - Retrieved Solution
Reseat the RAM sticks.
Did this solution work? Type yes or no: no
Enter the revised solution: replace the motherboard
Revised solution: replace the motherboard
RETAIN - New case added to the case base.
New Case Base length: 4